Lecture Notes: TwinCAT 3

Libraries and Code Reuse

March 17, 2026

Contents

1 The Purpose of Libraries **2**

2 Creating a Library Project **2**

3 Defining Library Properties **5**

4 Installation and Managed Libraries **6**

4.1 Installation Process 6

4.2 Library Categories 6

5 Referencing and Using a Library **8**

5.1 Instantiating 8

5.2 Namespace Usage 8

6 Hardware Mapping **11**

Quick Reference Card **12**

1 The Purpose of Libraries

As PLC projects grow in complexity or as a developer works across multiple machines, they often find themselves **rewriting or copy-pasting** the same logic. **Libraries** solve this by organizing code into modular packages that can be shared across multiple unrelated projects.

- **Code Reuse:** Standardized function blocks (FBs) and functions are stored in a common codebase.
- **Modularity:** Gaining the behavior of a library without having to implement it manually within every new program.
- **Independent Testing:** Logic can be modularized and tested separately from the main application.
- **Drawbacks:** It adds another software artifact to track and can lead to "dependency hell" in complex systems.

Approach	Advantages	Disadvantages
Copy-Paste	Simple, no dependencies	Hard to maintain, bug propagation
Library	Reusable, centralized updates	Version management complexity
Inline Code	Full control, no overhead	Code duplication, inconsistency

Table 1: Code Reuse Strategies Comparison

> **Note:** Libraries are especially valuable when working on multiple machines of the same type or when developing standardized components that will be used across different projects within an organization.

2 Creating a Library Project

In TwinCAT 3, a library is created as a separate **PLC project** that holds the "business logic" of the code.

- **Clean-up:** Library projects do not need the typical configurations for motion control, safety, or analytics unless they specifically control those things.
- **Non-Executable:** Unlike a standard PLC program, a library **does not require a MAIN program** because it is never executed on its own; it is compiled into other executables.

Example: Library Content Example

```
// ====================================================
// LIBRARY PROJECT STRUCTURE
// ====================================================
//
// MyIOLibrary (PLC Project - Library Type)
// |
// +-- POUs
// |    +-- FB_SensorHandler          <-- Function Block
// |    +-- FB_O5D100                 <-- Sensor-specific FB
```

```
// |    +-- FB_MotorController       <-- Motor control FB
// |    +-- FC_ScaleAnalog           <-- Helper function
// |
// +-- DUTs
// |    +-- ST_SensorData            <-- Data structure
// |    +-- E_SensorStatus           <-- Enumeration
// |
// +-- GVLs
// |    +-- GVL_LibraryConstants     <-- Library-wide constants
// |
// +-- [NO MAIN PROGRAM]            <-- Libraries have no MAIN!
//
// ====================================================

// FUNCTION BLOCK: FB_SensorHandler
// A generic sensor handler that can be reused across projects
// This FB is part of the library, not the main application

FUNCTION_BLOCK FB_SensorHandler
VAR_INPUT
    bEnable          : BOOL;          // Enable sensor reading
    nRawValue        : INT;           // Raw analog input (0-32767)
    nScaleMin        : INT := 0;      // Minimum scaled value
    nScaleMax        : INT := 100;    // Maximum scaled value
    tFilterTime      : TIME := T#100MS;  // Smoothing filter time
END_VAR

VAR_OUTPUT
    fScaledValue     : REAL;          // Scaled output value
    bValueValid      : BOOL;          // TRUE if reading is valid
    bSensorFault     : BOOL;          // TRUE if sensor error detected
    nStatus          : INT;           // Detailed status code
END_VAR

VAR
    // Internal processing variables
    fRawReal         : REAL;
    fFilteredValue   : REAL;
    fAlpha           : REAL := 0.1;   // Filter coefficient

    // Fault detection
    nLastRawValue    : INT;
    nFrozenCount     : INT;
    bInitialized     : BOOL := FALSE;
END_VAR

// Implementation
IF NOT bEnable THEN
    bValueValid := FALSE;
    nStatus := 0;    // Disabled
    RETURN;
END_IF

// Check for sensor fault (value frozen or out of range)
IF nRawValue = nLastRawValue THEN
    nFrozenCount := nFrozenCount + 1;
    IF nFrozenCount > 100 THEN
        bSensorFault := TRUE;
```

```
        nStatus := -1;  // Frozen value fault
    END_IF
ELSE
    nFrozenCount := 0;
    bSensorFault := FALSE;
END_IF
nLastRawValue := nRawValue;

// Check for out-of-range
IF nRawValue < 500 OR nRawValue > 32000 THEN
    bSensorFault := TRUE;
    nStatus := -2;  // Out of range fault
END_IF

// Scale the value
fRawReal := INT_TO_REAL(nRawValue);
fScaledValue := (fRawReal / 32767.0) *
                INT_TO_REAL(nScaleMax - nScaleMin) +
                INT_TO_REAL(nScaleMin);

// Apply simple low-pass filter
IF NOT bInitialized THEN
    fFilteredValue := fScaledValue;
    bInitialized := TRUE;
ELSE
    fFilteredValue := fAlpha * fScaledValue +
                      (1.0 - fAlpha) * fFilteredValue;
END_IF

fScaledValue := fFilteredValue;
bValueValid := NOT bSensorFault;
IF NOT bSensorFault THEN
    nStatus := 1;   // Normal operation
END_IF
```

```
// FUNCTION BLOCK: FB_O5D100
// IFM laser distance sensor - library version
// Note: I/O mapping (%I*) is NOT in the library
// The actual hardware connection is made in the main project

FUNCTION_BLOCK FB_O5D100
VAR_INPUT
    // Raw bytes passed IN from the main program
    // The main program handles the I/O mapping
    aRawData        : ARRAY[0..1] OF BYTE;
    bEnable         : BOOL := TRUE;
END_VAR

VAR_OUTPUT
    nDistance       : INT;          // Distance in mm
    bInValidRange   : BOOL;         // Measurement valid flag
    bError          : BOOL;         // Communication error
END_VAR

VAR
    wCombinedWord   : WORD;
END_VAR

```

```
IF NOT bEnable THEN
    bError := FALSE;
    bInValidRange := FALSE;
    RETURN;
END_IF

// Combine bytes and extract distance
wCombinedWord := SHL(TO_WORD(aRawData[0]), 8) OR TO_WORD(aRawData[1])
    ;
nDistance := TO_INT(SHR(wCombinedWord, 4));
bInValidRange := aRawData[1].0;
bError := FALSE;
```

```
// FUNCTION: FC_ScaleAnalog
// A simple helper function in the library
// Functions are stateless and reusable

FUNCTION FC_ScaleAnalog : REAL
VAR_INPUT
    nRawValue    : INT;       // Raw input (0-32767)
    fOutMin      : REAL;      // Output minimum
    fOutMax      : REAL;      // Output maximum
    nInMin       : INT := 0;      // Input minimum
    nInMax       : INT := 32767; // Input maximum
END_VAR

VAR
    fNormalized : REAL;
END_VAR

// Normalize input to 0.0 - 1.0 range
fNormalized := INT_TO_REAL(nRawValue - nInMin) /
               INT_TO_REAL(nInMax - nInMin);

// Limit to valid range
IF fNormalized < 0.0 THEN
    fNormalized := 0.0;
ELSIF fNormalized > 1.0 THEN
    fNormalized := 1.0;
END_IF

// Scale to output range
FC_ScaleAnalog := fNormalized * (fOutMax - fOutMin) + fOutMin;
```

Warning: Library projects should **never** contain I/O mappings (`AT %I*` or `AT %Q*`). The actual hardware connections are specific to each machine and must be handled in the main application project that references the library.

3 Defining Library Properties

To identify and manage a library, you must configure its properties.

- **Version Scheme:** A manual numbering system (e.g., `0.1.0.0`) that must be incremented for every new release.

- **Released Flag:** A protection mechanism that prevents further changes to the current version once set, forcing a version increment for future updates.
- **Namespace:** A critical feature to **avoid naming collisions**. If two different libraries have a function with the same name, the namespace allows the compiler to distinguish between them.
- **Global Version Structure:** Including this automatically creates a variable file that exposes the library's version information to the project referencing it.

Property	Example	Purpose
Title	MyIOLibrary	Display name in references
Version	1.2.0.0	Major.Minor.Build.Revision
Company	MyCompany AB	Organization identifier
Namespace	IO	Prefix for all library items
Released	TRUE/FALSE	Locks version for production

Table 2: Key Library Properties

Property	Value
Title	MyIOLibrary
Version	1.0.0.0
Default namespace	IO
Company	MyCompany AB
Released	☐ (unchecked during development)

Figure 1: Library Properties Dialog Settings

Tip: Use semantic versioning (MAJOR.MINOR.PATCH.BUILD) for your libraries. Increment MAJOR for breaking changes, MINOR for new features, PATCH for bug fixes, and BUILD for internal releases. This helps consumers of your library understand the impact of updates.

4 Installation and Managed Libraries

Once the code is ready, the library must be installed into the system to be accessible.

4.1 Installation Process

Right-clicking the project and selecting **"Save as library and install"** compiles the logic and places it in the **Managed Libraries** folder (typically `C:\TwinCAT\3.1\Components\Plc\Managed Libraries`).

4.2 Library Categories

By default, custom libraries appear in a "Miscellaneous" category. To organize them professionally, developers use a **category description file** (`.libcat.xml`) to create custom groups (e.g., "MyCompany AB").

Example: Library Category XML

```xml
<?xml version="1.0" encoding="utf-8"?>
<!--
    Library Category Definition File
    File name: OurLibraryCategory.libcat.xml

    Place this file in:
    C:\TwinCAT\3.1\Components\Plc\Managed Libraries\

    This creates a custom category in the "Add Library" dialog
    to organize your company's libraries professionally.
-->

<LibraryCategories
    xmlns="http://schemas.2010.2b.TcLc.Beckhoff.com">

    <!-- Main company category -->
    <LibraryCategory>
        <!-- Unique GUID - generate a new one for each category -->
        <!-- Use Visual Studio Tools > Create GUID or online generator -->
        <Id>{A1B2C3D4-E5F6-7890-ABCD-EF1234567890}</Id>

        <!-- Display name shown in the library browser -->
        <Name>MyCompany AB</Name>

        <!-- Optional: Category description -->
        <Description>
            Custom libraries developed by MyCompany AB
            for industrial automation projects.
        </Description>

        <!-- Optional: Version of the category file -->
        <Version>1.0.0.0</Version>
    </LibraryCategory>

    <!-- Sub-category for I/O related libraries -->
    <LibraryCategory>
        <Id>{B2C3D4E5-F6A7-8901-BCDE-F23456789012}</Id>
        <Name>MyCompany AB - I/O Libraries</Name>
        <Description>
            Sensor and actuator function blocks
        </Description>
        <!-- Parent reference creates hierarchy -->
        <ParentId>{A1B2C3D4-E5F6-7890-ABCD-EF1234567890}</ParentId>
    </LibraryCategory>

    <!-- Sub-category for Motion libraries -->
    <LibraryCategory>
        <Id>{C3D4E5F6-A7B8-9012-CDEF-345678901234}</Id>
        <Name>MyCompany AB - Motion</Name>
        <Description>
            Motion control and axis management
        </Description>
        <ParentId>{A1B2C3D4-E5F6-7890-ABCD-EF1234567890}</ParentId>
    </LibraryCategory>

```

```
    <!-- Sub-category for Utilities -->
    <LibraryCategory>
        <Id>{D4E5F6A7-B8C9-0123-DEFA-456789012345}</Id>
        <Name>MyCompany AB - Utilities</Name>
        <Description>
            Helper functions and common utilities
        </Description>
        <ParentId>{A1B2C3D4-E5F6-7890-ABCD-EF1234567890}</ParentId>
    </LibraryCategory>

</LibraryCategories>
```

```
<?xml version="1.0" encoding="utf-8"?>
<!--
    Minimal Library Category File
    For simple single-category setup
-->

<LibraryCategories
    xmlns="http://schemas.2010.2b.TcLc.Beckhoff.com">

    <LibraryCategory>
        <!-- Generate your own unique GUID -->
        <Id>{12345678-1234-1234-1234-123456789ABC}</Id>
        <Name>ACME Automation</Name>
    </LibraryCategory>

</LibraryCategories>
```

Note: After creating or modifying the `.libcat.xml` file, you must restart TwinCAT XAE (Visual Studio) for the changes to take effect. The new category will then appear in the "Add Library" dialog when adding references to your projects.

File Location	Purpose
`...\Managed Libraries\`	Installed library files (.library)
`...\Managed Libraries\*.libcat.xml`	Category definitions
`...\Repository\`	Library repository cache

Table 3: TwinCAT Library File Locations

5 Referencing and Using a Library

To use the library in a machine program (PLC_Program_1), it must be added as a **Reference**.

5.1 Instantiating

Once referenced, the library's FBs can be instantiated in the MAIN program or other POUs of the machine project.

5.2 Namespace Usage

To be explicit, you can call a function or FB using its namespace.

Example: Calling with Namespace

```
// PROGRAM: MAIN
// Main application program that USES the library
// This is in the machine project, NOT the library project

PROGRAM MAIN
VAR
    // ====================================================
    // INSTANTIATION WITHOUT NAMESPACE
    // Works if there are no naming conflicts
    // The compiler resolves the type from referenced libraries
    // ====================================================
    fbSensorSimple   : FB_SensorHandler;      // From MyIOLibrary
    fbDistSensor     : FB_O5D100;             // From MyIOLibrary

    // ====================================================
    // INSTANTIATION WITH EXPLICIT NAMESPACE
    // Recommended when:
    // - Multiple libraries have same-named types
    // - You want code to be self-documenting
    // - Avoiding ambiguity in large projects
    // ====================================================
    fbSensorExplicit : IO.FB_SensorHandler; // Namespace.TypeName
    fbMotor          : IO.FB_MotorController;

    // Example with potential naming conflict:
    // Both Tc2_Standard and MyIOLibrary might have CONCAT
    // Use namespace to specify which one

    // ====================================================
    // I/O VARIABLES - These are in the MAIN project
    // NOT in the library - hardware mapping is project-specific
    // ====================================================
    aDistanceSensorRaw AT %I* : ARRAY[0..1] OF BYTE;  // From EtherCAT
    nAnalogInput       AT %I* : INT;                  // Analog input
    bMotorOutput       AT %Q* : BOOL;                 // Motor contactor

    // Local variables
    fTemperature     : REAL;
    sMessage         : STRING(255);
    s1               : STRING := 'Sensor ';
    s2               : STRING := 'OK';
    sResult          : STRING(255);

END_VAR

// ====================================================
// CALLING LIBRARY FUNCTION BLOCKS
// Pass I/O data to the library FB instances
// ====================================================

// Method 1: Pass raw I/O to library FB
fbDistSensor(
    aRawData := aDistanceSensorRaw,     // Pass mapped I/O data
```

```
    bEnable := TRUE
);

// Use the outputs
IF fbDistSensor.bInValidRange THEN
    // Distance is valid, use fbDistSensor.nDistance
END_IF

// Method 2: Using the sensor handler
fbSensorExplicit(
    bEnable := TRUE,
    nRawValue := nAnalogInput,
    nScaleMin := 0,
    nScaleMax := 100
);
fTemperature := fbSensorExplicit.fScaledValue;

// ====================================================
// CALLING LIBRARY FUNCTIONS WITH NAMESPACE
// Use when function names might conflict
// ====================================================

// If your library has a CONCAT function that differs from standard:
sResult := IO.CONCAT(s1, s2);   // Explicitly use library version

// Standard library CONCAT (from Tc2_Standard):
// sResult := CONCAT(s1, s2);   // Uses default/first found

// ====================================================
// CALLING HELPER FUNCTIONS FROM LIBRARY
// ====================================================
fTemperature := IO.FC_ScaleAnalog(
    nRawValue := nAnalogInput,
    fOutMin := -20.0,
    fOutMax := 150.0
);
```

```
// Extended example: Multiple library references

PROGRAM MAIN
VAR
    // From IO library (namespace: IO)
    fbSensor1       : IO.FB_SensorHandler;
    fbSensor2       : IO.FB_O5D100;

    // From Motion library (namespace: Motion)
    // fbAxis1      : Motion.FB_SingleAxis;
    // fbHome       : Motion.FB_HomingSequence;

    // From Utilities library (namespace: Util)
    // fbLogger     : Util.FB_EventLogger;
    // fbBuffer     : Util.FB_DataBuffer;

    // Standard Beckhoff libraries (always available)
    fbTimer         : TON;              // From Tc2_Standard
    fbTrigger       : R_TRIG;           // From Tc2_Standard

    // I/O mappings - always in main project
```

```
    aSensor1Data     AT %I* : ARRAY[0..1] OF BYTE;
    aSensor2Data     AT %I* : ARRAY[0..3] OF BYTE;
    nAnalogIn1       AT %I* : INT;
    bDigitalOut1     AT %Q* : BOOL;

END_VAR

// Pass I/O to library instances
fbSensor1(
    bEnable := TRUE,
    nRawValue := nAnalogIn1
);

fbSensor2(
    aRawData := aSensor1Data,
    bEnable := TRUE
);

// Control output based on library FB results
bDigitalOut1 := fbSensor1.bValueValid AND fbSensor2.bInValidRange;
```

Tip: Always use explicit namespaces in production code, even when not strictly required. This makes the code self-documenting and prevents future issues if additional libraries with conflicting names are added to the project.

6 Hardware Mapping

While the library holds the logic (the "what"), the main PLC program handles the **I/O mapping** (the "where"). The program links the library instance's variables to the physical EtherCAT or fieldbus inputs and outputs.

Component	Location	Contains
Business Logic	Library	FBs, Functions, Algorithms
Data Structures	Library	DUTs, Enums, Structs
I/O Mapping	Main Project	AT %I*, AT %Q* variables
Hardware Config	Main Project	EtherCAT configuration
Task Assignment	Main Project	Cycle times, priorities

Table 4: Separation of Concerns: Library vs. Main Project

Library (Portable Code)	→	**Main Project** (Machine-Specific)
FB_O5D100 FB_SensorHandler FC_ScaleAnalog		I/O Mapping EtherCAT Config Task Assignment
↓		
Physical Hardware EtherCAT Terminals, Sensors, Actuators		

Figure 2: Library and Main Project Relationship

> ⚠ **Warning:** Never put `AT %I*` or `AT %Q*` declarations inside library code. This would make the library dependent on specific hardware configurations and defeat the purpose of code reuse. Always pass I/O data as inputs to library function blocks.

Quick Reference Card

Library Project Setup Checklist

1. Create new PLC Project (Standard or Library template)
2. Remove MAIN program (not needed for libraries)
3. Add POUs, DUTs, GVLs as needed
4. Set library properties (version, namespace, company)
5. Save as library and install
6. Create category XML file (optional)

Library Properties

```
// Key settings in Project Properties > PLC Project
Title:                MyIOLibrary
Version:              1.0.0.0
Default namespace:    IO
Company:              MyCompany AB
```

Using Libraries

```
// Add reference: Right-click References > Add Library

// Instantiate without namespace
fbSensor : FB_SensorHandler;

// Instantiate with namespace (recommended)
fbSensor : IO.FB_SensorHandler;

// Call function with namespace
fResult := IO.FC_ScaleAnalog(nRaw, 0.0, 100.0);
```

File Locations

`C:\TwinCAT\3.1\Components\Plc\Managed Libraries\`

Category XML Template

```xml
<?xml version="1.0" encoding="utf-8"?>
<LibraryCategories xmlns="http://schemas.2010.2b.TcLc.Beckhoff.com">
    <LibraryCategory>
        <Id>{YOUR-GUID-HERE}</Id>
        <Name>Your Category Name</Name>
    </LibraryCategory>
</LibraryCategories>
```

Last Updated: March 17, 2026